

CODE

climb_ga.py ****ALL OWN CODE****

```
import pybullet as p
import pybullet_data
import time
import numpy as np
import random
import creature
import population
import simulation
import genome
import math
import environment

#####
## Fitness Function ##
#####

def calculate_fitness(creature, steps, motors):
    start_pos = creature.start_position
    if not start_pos or not creature.last_position:
        return {
            "fitness": 0.0,
            "height_gained": 0.0,
            "is_stable": False
        }

    end_pos = creature.last_position
    end_orientation = creature.last_orientation
```

```

# Progress-based(Core climbing metrics)

height_gained = end_pos[2] - start_pos[2]

# Horizontal progress toward mountain center (0,0)

start_distance_to_mountain = ((start_pos[0])**2 + (start_pos[1])**2)**0.5
end_distance_to_mountain = ((end_pos[0])**2 + (end_pos[1])**2)**0.5

horizontal_progress = max(0, start_distance_to_mountain -
end_distance_to_mountain)

total_progress = (height_gained * 2.0) + (horizontal_progress * 2.0)

# Stability (Critical for real climbing)

euler_angles = p.getEulerFromQuaternion(end_orientation)
roll, pitch, yaw = euler_angles
tilt_magnitude = (roll**2 + pitch**2)**0.5

is_stable = tilt_magnitude < 0.5

# Stability scoring

if is_stable:
    stability_score = 1.0
elif tilt_magnitude < 1.0:
    stability_score = 0.5
else:
    stability_score = 0.0

# Efficiency (Time and energy considerations)

simulation_time = steps / 240.0

efficiency = total_progress / simulation_time if simulation_time > 0 else 0

```

```

# Terrain (Mountain-specific bonuses)

on_mountain_bonus = 0

if end_distance_to_mountain < 2.0 and stability_score > 0.5:
    on_mountain_bonus = 3.0

true_climbing_bonus = 0

if (end_distance_to_mountain < 3.0 and
    height_gained > 0.3 and
    stability_score > 0.5):
    true_climbing_bonus = height_gained * 15

# Collision with mountain

mountain_contact_bonus = 0
contact_time_bonus = 0

if creature.mountain_contacts > 0:
    mountain_contact_bonus = 5.0
    num_contacts = creature.mountain_contacts
    contact_time_bonus = min(num_contacts * 0.5, 3.0)

sustained_contact_bonus = 0

if creature.mountain_contacts > 10: # creature has been in contact for at
least 10 steps
    sustained_contact_bonus = creature.mountain_contacts * 0.1

# Weighted Fitness

fitness = (
    total_progress * 5.0 +
    stability_score * 4.0 +
    efficiency * 2.0 +
    on_mountain_bonus +
    true_climbing_bonus +
    mountain_contact_bonus +

```

```

        contact_time_bonus +
        sustained_contact_bonus
    )

# Cost of transport
total_motor_output = 0
for m in motors:
    total_motor_output += abs(m.get_output())

cost_of_transport = total_motor_output * 0.01
fitness -= cost_of_transport

if end_pos[2] > 1.2 and end_distance_to_mountain < 2.0:
    fitness += 5.0 # true climbing bonus

# Penalties
if stability_score == 0.0:
    fitness *= 0.2

if horizontal_progress < 0:
    fitness *= 0.7

return {
    "fitness": fitness,
    "height_gained": height_gained,
    "is_stable": is_stable
}

```

```
#####

## Evolution ##

#####

# Evolution parameters

GA_PARAMS = {
    'population_size': 100,
    'gene_count': 6,
    'generations': 300,
    'point_mutation_rate': 0.12,
    'point_mutation_amount': 0.05,
    'grow_mutation_rate': 0.15,
    'shrink_mutation_rate': 0.15,
    'simulation_steps': 2400,
    'elitism': True
}

def main():
    print("STARTING GENETIC ALGORITHM EVOLUTION")

    print(f"Evolution Parameters:")
    for key, value in GA_PARAMS.items():
        print(f"    {key}: {value}")

    # Create evolution population
    evo_pop = population.Population(pop_size=GA_PARAMS['population_size'],
                                    gene_count=GA_PARAMS['gene_count'])

    # Create fast headless simulation for evolution
    evo_sim = simulation.ThreadedSim(pool_size=4)
```

```

# Track evolution progress

all_time_best_fitness = -1
all_time_fittest_generation = -1
all_time_best_height = -1
all_time_hightest_generation = -1
total_stable_creatures = 0

print(f"\nStarting evolution for {GA_PARAMS['generations']} generations...")

for generation in range(GA_PARAMS['generations']):
    print(f"\n--- Generation {generation + 1} ---")

    # Evaluate all creatures using fast simulation
    print("Evaluating creatures...")
    evo_sim.eval_population(evo_pop, GA_PARAMS['simulation_steps'])

    # Calculate fitness scores and other stats
    fitness_details = [calculate_fitness(cr, GA_PARAMS['simulation_steps'],
    cr.get_motors()) for cr in evo_pop.creatures]

    fitness_scores = [d['fitness'] for d in fitness_details]
    heights = [d['height_gained'] for d in fitness_details]
    stable_count = sum(1 for d in fitness_details if d['is_stable'])
    total_stable_creatures += stable_count

    # Track statistics for this generation
    avg_fitness = np.mean(fitness_scores)
    best_fitness_this_gen = np.max(fitness_scores)
    best_height_this_gen = np.max(heights)

    if best_fitness_this_gen > all_time_best_fitness:
        all_time_best_fitness = best_fitness_this_gen
        all_time_fittest_generation = generation + 1

```

```

if best_height_this_gen > all_time_best_height:
    all_time_best_height = best_height_this_gen
    all_time_highesttest_generation = generation + 1

print(f"    Avg Fitness: {avg_fitness:.3f}")
print(f"    Best Fitness: {best_fitness_this_gen:.3f}")
print(f"    Best Height Gained: {best_height_this_gen:.3f}")
print(f"    Stable Creatures:
{stable_count}/{GA_PARAMS['population_size']}")

# Save elite creature

best_index = np.argmax(fitness_scores)
best_creature = evo_pop.creatures[best_index]

# Save to CSV for analysis

genome.Genome.to_csv(best_creature.dna, f"elite_gen_{generation + 1}.csv")

# Create next generation (if not last generation)

if generation < GA_PARAMS['generations'] - 1:
    print("    Creating next generation...")

    fitmap = population.Population.get_fitness_map(fitness_scores)
    new_creatures = []

    if GA_PARAMS['elitism']:
        new_creatures.append(best_creature)

    while len(new_creatures) < GA_PARAMS['population_size']:
        p1_ind = population.Population.select_parent(fitmap)
        p2_ind = population.Population.select_parent(fitmap)

        p1 = evo_pop.creatures[p1_ind]

```

```

p2 = evo_pop.creatures[p2_ind]

new_dna = genome.Genome.crossover(p1.dna, p2.dna)

new_dna = genome.Genome.point_mutate(new_dna,
GA_PARAMS['point_mutation_rate'], GA_PARAMS['point_mutation_amount'])

new_dna = genome.Genome.grow_mutate(new_dna,
GA_PARAMS['grow_mutation_rate'])

new_dna = genome.Genome.shrink_mutate(new_dna,
GA_PARAMS['shrink_mutation_rate'])

new_cr = creature.Creature(gene_count=len(new_dna))
new_cr.update_dna(new_dna)
new_creatures.append(new_cr)

evo_pop.creatures = new_creatures

print("EVOLUTIONARY RUN SUMMARY")
print(f"Population Size: {GA_PARAMS['population_size']}")
print(f"Genome Size (Gene Count): {GA_PARAMS['gene_count']}")
print(f"Point Mutation Rate: {GA_PARAMS['point_mutation_rate']}")
print(f"Grow/Shrink Rate:
{GA_PARAMS['grow_mutation_rate']}/{GA_PARAMS['shrink_mutation_rate']}")
print(f"Generations: {GA_PARAMS['generations']}")
print(f"Simulation Time per Creature:
{GA_PARAMS['simulation_steps']/240.0:.2f}s")
print(f"Average Total Fitness of the Run: {np.mean([d['fitness'] for d in
fitness_details]):.3f}")
print(f"Best Total Fitness of the Run: {all_time_best_fitness:.3f} (Generation
{all_time_fittest_generation})")
print(f"Best Height Gained: {all_time_best_height:.3f} (Generation
{all_time_highest_generation})")
print(f"Percentage of Stable Creatures: {(total_stable_creatures /
(GA_PARAMS['population_size'] * GA_PARAMS['generations'])) * 100:.2f}%")
if __name__ == '__main__':
    main()

```



```
creature.py **OWN CODE IN GREEN**
```

```
import genome

from xml.dom.minidom import getDOMImplementation
from enum import Enum
import numpy as np

class MotorType(Enum):
    PULSE = 1
    SINE = 2
    SAWTOOTH = 3

class Motor:
    def __init__(self, control_waveform, control_amp, control_freq,
control_smoothing, control_phase_offset):
        if control_waveform <= 0.33:
            self.motor_type = MotorType.PULSE
        elif control_waveform <= 0.66:
            self.motor_type = MotorType.SINE
        else:
            self.motor_type = MotorType.SAWTOOTH
        self.amp = control_amp
        self.freq = control_freq
        self.smoothing = control_smoothing
        self.phase = control_phase_offset
        self.prev_output = 0

    def get_output(self):
        self.phase = (self.phase + self.freq) % (np.pi * 2)

        raw_output = 0
        if self.motor_type == MotorType.PULSE:
            if self.phase < np.pi:
```

```

        raw_output = 1
    else:
        raw_output = -1

    if self.motor_type == MotorType.SINE:
        raw_output = np.sin(self.phase)

    if self.motor_type == MotorType.SAWTOOTH:
        raw_output = (self.phase / (2 * np.pi)) * 2 - 1

    smoothed_output = (1 - self.smoothing) * raw_output + self.smoothing *
self.prev_output

    self.prev_output = smoothed_output

    return smoothed_output

```

```

class Creature:

```

```

    def __init__(self, gene_count):
        self.spec = genome.Genome.get_gene_spec()
        self.dna = genome.Genome.get_random_genome(len(self.spec), gene_count)
        self.flat_links = None
        self.exp_links = None
        self.motors = None
        self.start_position = [0,0,0]
        self.last_position = [0,0,0]
        self.last_orientation = [0,0,0,1]
        self.mountain_contacts = 0

```

```

    def get_flat_links(self):
        if self.flat_links == None:
            gdicts = genome.Genome.get_genome_dicts(self.dna, self.spec)
            self.flat_links = genome.Genome.genome_to_links(gdicts)
        return self.flat_links

```

```

def get_expanded_links(self):
    self.get_flat_links()
    if self.exp_links is not None:
        return self.exp_links

    exp_links = [self.flat_links[0]]
    genome.Genome.expandLinks(self.flat_links[0],
                               self.flat_links[0].name,
                               self.flat_links,
                               exp_links)

    self.exp_links = exp_links
    return self.exp_links

def to_xml(self):
    self.get_expanded_links()
    domimpl = getDOMImplementation()
    adom = domimpl.createDocument(None, "start", None)
    robot_tag = adom.createElement("robot")
    for link in self.exp_links:
        robot_tag.appendChild(link.to_link_element(adom))
    first = True
    for link in self.exp_links:
        if first:
            first = False
            continue
        robot_tag.appendChild(link.to_joint_element(adom))
    robot_tag.setAttribute("name", "devbot")
    return '<?xml version="1.0"?>' + robot_tag.toprettyxml()

def get_motors(self):
    self.get_expanded_links()

```

```

if self.motors == None:
    motors = []
    for i in range(1, len(self.exp_links)):
        l = self.exp_links[i]
        m = Motor(l.control_waveform, l.control_amp, l.control_freq,
l.control_smoothing, l.control_phase_offset)
        motors.append(m)
    self.motors = motors
return self.motors

def update_position(self, pos):
    if self.start_position == None:
        self.start_position = pos
    else:
        self.last_position = pos

def get_distance_travelled(self):
    if self.start_position is None or self.last_position is None:
        return 0
    p1 = np.asarray(self.start_position)
    p2 = np.asarray(self.last_position)
    dist = np.linalg.norm(p1-p2)
    return dist

def update_dna(self, dna):
    self.dna = dna
    self.flat_links = None
    self.exp_links = None
    self.motors = None
    self.start_position = [0,0,0]
    self.last_position = [0,0,0]
    self.last_orientation = [0,0,0,1]
    self.mountain_contacts = 0

```

```
environment.py **IMPORTED MOUNTAIN & ARENA FROM CW-ENV.T.PY**
```

```
import pybullet as p

import math

import random


def make_mountain(num_rocks=100, max_size=0.25, arena_size=10, mountain_height=5,
physicsClientId=0):

    def gaussian(x, y, sigma=arena_size/4):

        """Return the height of the mountain at position (x, y) using a Gaussian
        function."""

        return mountain_height * math.exp(-((x**2 + y**2) / (2 * sigma**2)))

    for _ in range(num_rocks):

        x = random.uniform(-1 * arena_size/2, arena_size/2)

        y = random.uniform(-1 * arena_size/2, arena_size/2)

        z = gaussian(x, y) # Height determined by the Gaussian function


        # Adjust the size of the rocks based on height. Higher rocks (closer to
        the peak) will be smaller.

        size_factor = 1 - (z / mountain_height)

        size = random.uniform(0.1, max_size) * size_factor

        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14),
        random.uniform(0, 3.14), random.uniform(0, 3.14)])

        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size,
        size], physicsClientId=physicsClientId)

        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size,
        size], rgbColor=[0.5, 0.5, 0.5, 1], physicsClientId=physicsClientId)

        rock_body = p.createMultiBody(baseMass=0,
        baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual,
        basePosition=[x, y, z], baseOrientation=orientation,
        physicsClientId=physicsClientId)


def make_rocks(num_rocks=100, max_size=0.25, arena_size=10, physicsClientId=0):

    for _ in range(num_rocks):

        x = random.uniform(-1 * arena_size/2, arena_size/2)

        y = random.uniform(-1 * arena_size/2, arena_size/2)

        z = 0.5 # Adjust based on your needs
```

```

        size = random.uniform(0.1,max_size)

        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14),
random.uniform(0, 3.14), random.uniform(0, 3.14)])

        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size,
size], physicsClientId=physicsClientId)

        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size,
size], rgbColor=[0.5, 0.5, 0.5, 1], physicsClientId=physicsClientId)

        rock_body = p.createMultiBody(baseMass=0,
baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual,
basePosition=[x, y, z], baseOrientation=orientation,
physicsClientId=physicsClientId)

def make_arena(arena_size=10, wall_height=1, physicsClientId=0):

    wall_thickness = 0.5

    floor_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, arena_size/2, wall_thickness],
physicsClientId=physicsClientId)

    floor_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, arena_size/2, wall_thickness], rgbColor=[1, 1, 0, 1],
physicsClientId=physicsClientId)

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=floor_collision_shape,
baseVisualShapeIndex=floor_visual_shape, basePosition=[0, 0, -wall_thickness],
physicsClientId=physicsClientId)

    wall_collision_shape_ew = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, wall_thickness/2, wall_height/2],
physicsClientId=physicsClientId)

    wall_visual_shape_ew = p.createVisualShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, wall_thickness/2, wall_height/2], rgbColor=[0.7, 0.7,
0.7, 1], physicsClientId=physicsClientId) # Gray walls

    # Create east/west walls

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ew,
baseVisualShapeIndex=wall_visual_shape_ew, basePosition=[0, arena_size/2,
wall_height/2], physicsClientId=physicsClientId)

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ew,
baseVisualShapeIndex=wall_visual_shape_ew, basePosition=[0, -arena_size/2,
wall_height/2], physicsClientId=physicsClientId)

    wall_collision_shape_ns = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[wall_thickness/2, arena_size/2, wall_height/2],
physicsClientId=physicsClientId)

```

```

    wall_visual_shape_ns = p.createVisualShape(shapeType=p.GEOM_BOX,
halfExtents=[wall_thickness/2, arena_size/2, wall_height/2], rgbColor=[0.7, 0.7,
0.7, 1], physicsClientId=physicsClientId) # Gray walls

```

```

# Create north/south walls

```

```

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ns,
baseVisualShapeIndex=wall_visual_shape_ns, basePosition=[arena_size/2, 0,
wall_height/2], physicsClientId=physicsClientId)

```

```

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ns,
baseVisualShapeIndex=wall_visual_shape_ns, basePosition=[-arena_size/2, 0,
wall_height/2], physicsClientId=physicsClientId)

```

```

genome.py **OWN CODE IN GREEN**

```

```

import numpy as np

```

```

import copy

```

```

import random

```

```

class Genome():

```

```

    @staticmethod

```

```

    def get_random_gene(length):

```

```

        gene = np.array([np.random.random() for i in range(length)])

```

```

        return gene

```

```

    @staticmethod

```

```

    def get_random_genome(gene_length, gene_count):

```

```

        genome = [Genome.get_random_gene(gene_length) for i in range(gene_count)]

```

```

        return genome

```

```

    @staticmethod

```

```

    def get_gene_spec():

```

```

        gene_spec = {"link-shape":{"scale":1},

```

```

                    "link-length": {"scale":2},

```

```

                    "link-radius": {"scale":0.5},

```

```

                    "link-recurrence": {"scale":3},

```

```

                    "link-mass": {"scale":0.5},

```

```

        "joint-type": {"scale":1},
        "joint-parent":{"scale":1},
        "joint-axis-xyz": {"scale":1},
        "joint-origin-rpy-1":{"scale":np.pi * 4},
        "joint-origin-rpy-2":{"scale":np.pi * 4},
        "joint-origin-rpy-3":{"scale":np.pi * 4},
        "joint-origin-xyz-1":{"scale":0.5},
        "joint-origin-xyz-2":{"scale":0.5},
        "joint-origin-xyz-3":{"scale":0.5},
        "control-waveform":{"scale":1},
        "control-amp":{"scale":1.0},
        "control-freq":{"scale":1},
        "control-smoothing": {"scale": 0.1},
        "taper": {"scale": 1.0},
        "control_phase_offset": {"scale": 2*np.pi}
    }
    ind = 0
    for key in gene_spec.keys():
        gene_spec[key]["ind"] = ind
        ind = ind + 1
    return gene_spec

```

```

@staticmethod
def get_gene_dict(gene, spec):
    gdict = {}
    for key in spec:
        ind = spec[key]["ind"]
        scale = spec[key]["scale"]
        gdict[key] = gene[ind] * scale
    return gdict

```



```

@staticmethod
def get_genome_dicts(genome, spec):
    gdicts = []
    for gene in genome:
        gdicts.append(Genome.get_gene_dict(gene, spec))
    return gdicts

@staticmethod
def expandLinks(parent_link, uniq_parent_name, flat_links, exp_links):
    children = [l for l in flat_links if l.parent_name == parent_link.name]
    sibling_ind = 1
    for c in children:
        for r in range(int(c.recur)):
            sibling_ind = sibling_ind + 1
            c_copy = copy.copy(c)
            c_copy.parent_name = uniq_parent_name
            uniq_name = c_copy.name + str(len(exp_links))
            #print("exp: ", c.name, " -> ", uniq_name)
            c_copy.name = uniq_name
            c_copy.sibling_ind = sibling_ind
            exp_links.append(c_copy)
            assert c.parent_name != c.name, "Genome::expandLinks: link joined
to itself: " + c.name + " joins " + c.parent_name
            Genome.expandLinks(c, uniq_name, flat_links, exp_links)

@staticmethod
def genome_to_links(gdicts):
    links = []
    link_ind = 0
    parent_names = [str(link_ind)]
    for gdict in gdicts:
        link_name = str(link_ind)
        parent_ind = gdict["joint-parent"] * len(parent_names)

```

```

        assert parent_ind < len(parent_names), "genome.py: parent ind too
high: " + str(parent_ind) + "got: " + str(parent_names)

        parent_name = parent_names[int(parent_ind)]
        #print("available parents: ", parent_names, "chose", parent_name)
        recur = gdict["link-recurrence"]
        link = URDFLink(name=link_name,
                        parent_name=parent_name,
                        recur=recur+1,
                        link_length=gdict["link-length"],
                        link_radius=gdict["link-radius"],
                        link_shape=gdict["link-shape"],
                        link_mass=gdict["link-mass"],
                        joint_type=gdict["joint-type"],
                        joint_parent=gdict["joint-parent"],
                        joint_axis_xyz=gdict["joint-axis-xyz"],
                        joint_origin_rpy_1=gdict["joint-origin-rpy-1"],
                        joint_origin_rpy_2=gdict["joint-origin-rpy-2"],
                        joint_origin_rpy_3=gdict["joint-origin-rpy-3"],
                        joint_origin_xyz_1=gdict["joint-origin-xyz-1"],
                        joint_origin_xyz_2=gdict["joint-origin-xyz-2"],
                        joint_origin_xyz_3=gdict["joint-origin-xyz-3"],
                        control_waveform=gdict["control-waveform"],
                        control_amp=gdict["control-amp"],
                        control_freq=gdict["control-freq"],
                        control_smoothing=gdict["control-smoothing"],
                        taper=gdict["taper"],
                        control_phase_offset=gdict["control_phase_offset"])
        links.append(link)

        if link_ind != 0: # don't re-add the first link
            parent_names.append(link_name)

        link_ind = link_ind + 1

```

```

# now just fix the first link so it links to nothing
links[0].parent_name = "None"
return links

```

```

@staticmethod
def crossover(g1, g2):
    x1 = random.randint(0, len(g1)-1)
    x2 = random.randint(0, len(g2)-1)
    g3 = np.concatenate((g1[x1:], g2[x2:]))
    if len(g3) > len(g1):
        g3 = g3[0:len(g1)]
    return g3

```

```

@staticmethod
def point_mutate(genome, rate, amount):
    new_genome = copy.copy(genome)
    for gene in new_genome:
        for i in range(len(gene)):
            if random.random() < rate:
                gene[i] += amount
            if gene[i] >= 1.0:
                gene[i] = 0.9999
            if gene[i] < 0.0:
                gene[i] = 0.0
    return new_genome

```

```

@staticmethod
def shrink_mutate(genome, rate):
    if len(genome) == 1:
        return copy.copy(genome)
    if random.random() < rate:
        ind = random.randint(0, len(genome)-1)

```

```

        new_genome = np.delete(genome, ind, 0)

        return new_genome

    else:

        return copy.copy(genome)

    @staticmethod

    def grow_mutate(genome, rate):

        MAX_GENOME_LENGTH = 12 #limit grow

        if len(genome) >= MAX_GENOME_LENGTH:

            return copy.copy(genome)

        if random.random() < rate:

            gene = Genome.get_random_gene(len(genome[0]))

            new_genome = copy.copy(genome)

            new_genome = np.append(new_genome, [gene], axis=0)

            return new_genome

        else:

            return copy.copy(genome)

    @staticmethod

    def to_csv(dna, csv_file):

        csv_str = ""

        for gene in dna:

            for val in gene:

                csv_str = csv_str + str(val) + ","

            csv_str = csv_str + '\n'

        with open(csv_file, 'w') as f:

            f.write(csv_str)

    @staticmethod

    def from_csv(filename):

        csv_str = ''

```

```

with open(filename) as f:
    csv_str = f.read()
dna = []
lines = csv_str.split('\n')
for line in lines:
    vals = line.split(',')
    gene = [float(v) for v in vals if v != '']
    if len(gene) > 0:
        dna.append(gene)
return dna

```

```

class URDFLink:
    def __init__(self, name, parent_name, recur,
        link_length=0.1,
        link_radius=0.1,
        link_shape=0.1,
        link_mass=0.1,
        joint_type=0.1,
        joint_parent=0.1,
        joint_axis_xyz=0.1,
        joint_origin_rpy_1=0.1,
        joint_origin_rpy_2=0.1,
        joint_origin_rpy_3=0.1,
        joint_origin_xyz_1=0.1,
        joint_origin_xyz_2=0.1,
        joint_origin_xyz_3=0.1,
        control_waveform=0.1,
        control_amp=0.1,
        control_freq=0.1,
        control_smoothing=0.1,
        taper=0.1,
        control_phase_offset=0.1):

```

```

self.name = name
self.parent_name = parent_name
self.recur = recur
self.link_length=link_length
self.link_radius=link_radius
self.link_shape=link_shape
self.link_mass=link_mass
self.joint_type=joint_type
self.joint_parent=joint_parent
self.joint_axis_xyz=joint_axis_xyz
self.joint_origin_rpy_1=joint_origin_rpy_1
self.joint_origin_rpy_2=joint_origin_rpy_2
self.joint_origin_rpy_3=joint_origin_rpy_3
self.joint_origin_xyz_1=joint_origin_xyz_1
self.joint_origin_xyz_2=joint_origin_xyz_2
self.joint_origin_xyz_3=joint_origin_xyz_3
self.control_waveform=control_waveform
self.control_amp=control_amp
self.control_freq=control_freq
self.control_smoothing=control_smoothing
self.taper=taper
self.control_phase_offset = control_phase_offset
self.sibling_ind = 1

```

```

def to_link_element(self, adom):
    link_tag = adom.createElement("link")
    link_tag.setAttribute("name", self.name)
    vis_tag = adom.createElement("visual")
    geom_tag = adom.createElement("geometry")

    if self.link_shape < 0.33:
        shape_tag = adom.createElement("box")

```

```

        shape_tag.setAttribute("size", str(self.link_length) + " " +
str(self.link_radius * self.taper) + " " + str(self.link_radius * self.taper))

    elif self.link_shape < 0.66:

        shape_tag = adom.createElement("capsule")

        shape_tag.setAttribute("length", str(self.link_length))

        shape_tag.setAttribute("radius", str(self.link_radius))
    else:

        shape_tag = adom.createElement("sphere")

        shape_tag.setAttribute("radius", str(self.link_radius))

geom_tag.appendChild(shape_tag)
vis_tag.appendChild(geom_tag)


coll_tag = adom.createElement("collision")
c_geom_tag = adom.createElement("geometry")


if self.link_shape < 0.33:

    c_shape_tag = adom.createElement("box")

    c_shape_tag.setAttribute("size", str(self.link_length) + " " +
str(self.link_radius * self.taper) + " " + str(self.link_radius * self.taper))

    elif self.link_shape < 0.66:

        c_shape_tag = adom.createElement("capsule")

        c_shape_tag.setAttribute("length", str(self.link_length))

        c_shape_tag.setAttribute("radius", str(self.link_radius))
    else:

        c_shape_tag = adom.createElement("sphere")

        c_shape_tag.setAttribute("radius", str(self.link_radius))

c_geom_tag.appendChild(c_shape_tag)
coll_tag.appendChild(c_geom_tag)


inertial_tag = adom.createElement("inertial")
mass_tag = adom.createElement("mass")

```

```

# pi r^2 * height
mass = np.pi * (self.link_radius * self.link_radius) * self.link_length
mass_tag.setAttribute("value", str(mass))
inertia_tag = adom.createElement("inertia")
# <inertia ixx="0.0003" iyy="0.0003" izz="0.0003" ixy="0" ixz="0"
iyz="0"/>
inertia_tag.setAttribute("ixx", "0.03")
inertia_tag.setAttribute("iyy", "0.03")
inertia_tag.setAttribute("izz", "0.03")
inertia_tag.setAttribute("ixy", "0")
inertia_tag.setAttribute("ixz", "0")
inertia_tag.setAttribute("iyz", "0")
inertial_tag.appendChild(mass_tag)
inertial_tag.appendChild(inertia_tag)

link_tag.appendChild(vis_tag)
link_tag.appendChild(coll_tag)
link_tag.appendChild(inertial_tag)

return link_tag

def to_joint_element(self, adom):
    joint_tag = adom.createElement("joint")
    joint_tag.setAttribute("name", self.name + "_to_" + self.parent_name)
    joint_tag.setAttribute("type", "revolute")
    parent_tag = adom.createElement("parent")
    parent_tag.setAttribute("link", self.parent_name)
    child_tag = adom.createElement("child")
    child_tag.setAttribute("link", self.name)
    axis_tag = adom.createElement("axis")
    if self.joint_axis_xyz <= 0.33:
        axis_tag.setAttribute("xyz", "1 0 0")
    if self.joint_axis_xyz > 0.33 and self.joint_axis_xyz <= 0.66:

```



```

        axis_tag.setAttribute("xyz", "0 1 0")
    if self.joint_axis_xyz > 0.66:
        axis_tag.setAttribute("xyz", "0 0 1")

    limit_tag = adom.createElement("limit")
    # effort upper lower velocity
    limit_tag.setAttribute("effort", "1")
    limit_tag.setAttribute("upper", "-3.1415")
    limit_tag.setAttribute("lower", "3.1415")
    limit_tag.setAttribute("velocity", "1")
    # <origin rpy="0 0 0" xyz="0 0.5 0"/>
    orig_tag = adom.createElement("origin")

    rpy1 = self.joint_origin_rpy_1 * self.sibling_ind
    rpy = str(rpy1) + " " + str(self.joint_origin_rpy_2) + " " +
    str(self.joint_origin_rpy_3)

    orig_tag.setAttribute("rpy", rpy)
    xyz = str(self.joint_origin_xyz_1) + " " + str(self.joint_origin_xyz_2) +
    " " + str(self.joint_origin_xyz_3)
    orig_tag.setAttribute("xyz", xyz)

    joint_tag.appendChild(parent_tag)
    joint_tag.appendChild(child_tag)
    joint_tag.appendChild(axis_tag)
    joint_tag.appendChild(limit_tag)
    joint_tag.appendChild(orig_tag)
    return joint_tag

```

population.py ****OWN CODE IN GREEN****

```
import creature

import numpy as np

class Population:

    def __init__(self, pop_size, gene_count):
        self.creatures = [creature.Creature(
                                gene_count=gene_count)
                            for i in range(pop_size)]

    @staticmethod
    def get_fitness_map(fits):
        fitmap = []
        total = 0
        min_fit = np.min(fits)
        for f in fits:
            total = total + f - min_fit
            fitmap.append(total)
        return fitmap

    @staticmethod
    def select_parent(fitmap):
        r = np.random.rand() # 0-1
        r = r * fitmap[-1]
        for i in range(len(fitmap)):
            if r <= fitmap[i]:
                return i
```

realtime_from_csv.py ****OWN CODE IN GREEN****

```
import os
import genome
import sys
import creature
import pybullet as p
import time
import random
import numpy as np

## ... usual starter code to create a sim and floor
import environment

def main(csv_file):
    assert os.path.exists(csv_file), "Tried to load " + csv_file + " but it does not exist"

    p.connect(p.GUI)
    p.setPhysicsEngineParameter(enableFileCaching=0)
    p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
    p.resetDebugVisualizerCamera(cameraDistance=15, cameraYaw=45, cameraPitch=-30, cameraTargetPosition=[0,0,0])
    p.setGravity(0, 0, -10)

    # Create the environment
    arena_size = 20
    environment.make_arena(arena_size=arena_size)
    p.setAdditionalSearchPath('shapes/')

    mountain = p.loadURDF("gaussian_pyramid.urdf", (0, 0, 0.5),
    p.getQuaternionFromEuler((0,0,0)), useFixedBase=1)
```

generate a random creature

```
dna = genome.Genome.from_csv(csv_file)
cr = creature.Creature(gene_count=len(dna))
cr.update_dna(dna)
# save it to XML
with open('test.urdf', 'w') as f:
    f.write(cr.to_xml())
# load it into the sim
rob1 = p.loadURDF('test.urdf')
# place it at the start
p.resetBasePositionAndOrientation(rob1, [6, 0, 0.5], [0, 0, 0, 1])
start_pos, orn = p.getBasePositionAndOrientation(rob1)
```

iterate

```
elapsed_time = 0
wait_time = 1.0/240 # seconds
total_time = 30 # seconds
step = 0
while True:
    p.stepSimulation()
    step += 1
    if step % 24 == 0:
        motors = cr.get_motors()
        assert len(motors) == p.getNumJoints(rob1), "Something went wrong"
        for jid in range(p.getNumJoints(rob1)):
            mode = p.VELOCITY_CONTROL
            vel = motors[jid].get_output()
            p.setJointMotorControl2(rob1,
                                    jid,
                                    controlMode=mode,
                                    targetVelocity=vel)
        new_pos, orn = p.getBasePositionAndOrientation(rob1)
```

```

        #print(new_pos)

        dist_moved = np.linalg.norm(np.asarray(start_pos) -
np.asarray(new_pos))

        time.sleep(wait_time)

        elapsed_time += wait_time

        if elapsed_time > total_time:

            break

if __name__ == "__main__":
    assert len(sys.argv) == 2, "Usage: python playback_test.py csv_filename"
    main(sys.argv[1])

```

simulation.py ****OWN CODE IN GREEN****

```

import pybullet as p
from multiprocessing import Pool

class Simulation:

    def __init__(self, sim_id=0):
        self.sim_id = sim_id
        self.physicsClientId = -1

    def run_creature(self, cr, iterations=2400):
        try:

            self.physicsClientId = p.connect(p.DIRECT)

            pid = self.physicsClientId
            p.resetSimulation(physicsClientId=pid)
            p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)

            p.setGravity(0, 0, -10, physicsClientId=pid)

            import environment

            arena_size = 20

            environment.make_arena(arena_size=arena_size, physicsClientId=pid)

```

```

mountain_position = (0, 0, 0.5)
mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
p.setAdditionalSearchPath('shapes/', physicsClientId=pid)
mountain = p.loadURDF("gaussian_pyramid.urdf",
                      mountain_position,
                      mountain_orientation,
                      useFixedBase=1,
                      physicsClientId=pid)

cr.start_position = [6,0,0.5]
xml_file = 'temp' + str(self.sim_id) + '.urdf'
xml_str = cr.to_xml()
with open(xml_file, 'w') as f:
    f.write(xml_str)

cid = p.loadURDF(xml_file, physicsClientId=pid)
if cid < 0:
    raise Exception("Failed to load URDF")

p.resetBasePositionAndOrientation(cid, cr.start_position, [0, 0, 0,
1], physicsClientId=pid)

for step in range(iterations):
    p.stepSimulation(physicsClientId=pid)
    if step % 24 == 0:
        self.update_motors(cid=cid, cr=cr)

        pos, orn = p.getBasePositionAndOrientation(cid,
physicsClientId=pid)
        cr.update_position(pos)

cr.last_orientation = orn

```

```

        cr.last_position = pos

        cr.mountain_contacts = len(p.getContactPoints(bodyA=cid,
bodyB=mountain, physicsClientId=pid))

    except Exception as e:

        # This handles creatures that are not valid URDFs

        cr.last_position = cr.start_position

        cr.last_orientation = [0,0,0,1]

        cr.mountain_contacts = 0

    finally:

        if self.physicsClientId >= 0:

            p.disconnect(self.physicsClientId)

```

```

def update_motors(self, cid, cr):
    """
    cid is the id in the physics engine
    cr is a creature object
    """

    for jid in range(p.getNumJoints(cid,
                                physicsClientId=self.physicsClientId)):

        m = cr.get_motors()[jid]

        p.setJointMotorControl2(cid, jid,
                                controlMode=p.VELOCITY_CONTROL,
                                targetVelocity=m.get_output(),
                                force = 5,
                                physicsClientId=self.physicsClientId)

```

You can add this to the Simulation class:

```

def eval_population(self, pop, iterations):

    for cr in pop.creatures:

        self.run_creature(cr, 2400)

```

```

class ThreadedSim():
    def __init__(self, pool_size):
        self.sims = [Simulation(i) for i in range(pool_size)]

    @staticmethod
    def static_run_creature(sim, cr, iterations):
        sim.run_creature(cr, iterations)
        return cr

    def eval_population(self, pop, iterations):
        """
        pop is a Population object
        iterations is frames in pybullet to run for at 240fps
        """
        pool_args = []
        start_ind = 0
        pool_size = len(self.sims)
        while start_ind < len(pop.creatures):
            this_pool_args = []
            for i in range(start_ind, start_ind + pool_size):
                if i == len(pop.creatures):# the end
                    break
                # work out the sim ind
                sim_ind = i % len(self.sims)
                this_pool_args.append([
                    self.sims[sim_ind],
                    pop.creatures[i],
                    iterations]
                )
            pool_args.append(this_pool_args)
            start_ind = start_ind + pool_size

```



```
new_creatures = []
for pool_argset in pool_args:
    with Pool(pool_size) as p:
        # it works on a copy of the creatures, so receive them
        creatures = p.starmap(ThreadedSim.static_run_creature,
pool_argset)
        # and now put those creatures back into the main
        # self.creatures array
        new_creatures.extend(creatures)
pop.creatures = new_creatures
```

****END****